23CSE455 Design Patterns

Case Study Report

Design Pattern Analysis and Evolution in the PlantUML Diagram Generation System

Team A17

Roll number	Student name	Batch
AM.SC.U4CSE23064	Sathvik Pasuvula	CSE A
AM.SC.U4CSE23142	P Sai Satvik Chowdary	CSE A

Application: PlantUML, class-diagram family
Selected baseline: v1.2026.8
Date: 2026-10-11
Submission repository: 23CSE455_DP_CaseStudy

Scope of this submission

Five source-verified GoF collaborations, three implemented changes, and equivalent four-language examples. Source snapshots, patches, tests, raw timings and datasets accompany this report. Human review, contribution confirmation, faculty approval and repository merge are explicitly separated from automated verification.

Contents

1 Application and architecture **2**

1.1 Architecture and a concrete input trace 2

2 Part 1: Pattern discovery **4**

2.1 Method and counting rules 4

2.2 Command: Class diagram command dispatch 5

2.3 Factory Method: Class diagram product creation 7

2.4 Strategy: Class diagram layout backend selection 9

2.5 Composite: Class header text-block composition 11

2.6 Adapter: VizJs output-stream bridge 13

3 Part 2: Evolution and refactoring **15**

3.1 CH01: Explicit review status on a class declaration 15

3.2 CH02: Configurable review styling policy 16

3.3 CH03: Optional SVG metadata and a local refactoring 16

3.4 Snapshots, metrics and pattern effects 18

4 Part 3: Cross-language comparison **20**

4.1 Selected problems and equivalent behavior 20

4.2 Static and process measurements 20

4.3 Structure, coupling and language features 21

5 LLM records and human verification **22**

6 Findings, contributions and references **22**

6.1 Findings and limitations 22

6.2 Contributions 23

6.3 References 23

7 Team folder and datasets **24**

1 Application and architecture

PlantUML converts a textual description into a diagram. This study follows class declarations and an association through command recognition, model construction, layout, styling and image export. Its purpose is to test pattern claims against working source, then observe what actually changes when requirements evolve.

Original source. The official baseline is PlantUML v1.2026.8, accessed 2026-10-11. The peeled commit is `149874a1bb2b64c42889d7178cb89f6647ac14bc`. The annotated tag object is `b531f7b00ebf733d9079a7b0a0180c64ded21ef0`; these hashes are not interchangeable. The inspected source distribution uses GPL v3-or-later notices. Upstream license files accompany the copied source. Faculty approval was not supplied in this interaction.

Environment. The runs used macOS 27.0.1 on arm64, OpenJDK 25.0.2, Gradle 9.3.1 and Graphviz 16.1.0. Production Java was compiled with release 11. The snippets used Python 3.14.2, Node.js 22.19.0 and Apple clang 21.0.0 with C++17. There is no database in the inspected path. The baseline dependency catalog pins JUnit 6.1.3, ELK 0.12.0, TeaVM 0.14.1, JLaTeXMath 1.0.7 and OpenPDF 3.0.5. The backend-specific dependencies are not all needed by the small class example.

Scope. The inventory investigates the five pattern types named in the brief: Command, Factory Method, Strategy, Composite and Adapter. It covers class commands, their receiver objects, layout backend selection, class-header text blocks and the VizJs output bridge. It is a representative collaboration study, not a census of all PlantUML commands or every TextBlock implementation. Sequence/activity families, web-server internals, browser execution and optional VizJs runtime installation are excluded from the integration experiment. All mapped participant classes are recorded in `data/patterns.csv`; the local source subset preserves upstream paths under `Part1/code/`.

1.1 Architecture and a concrete input trace

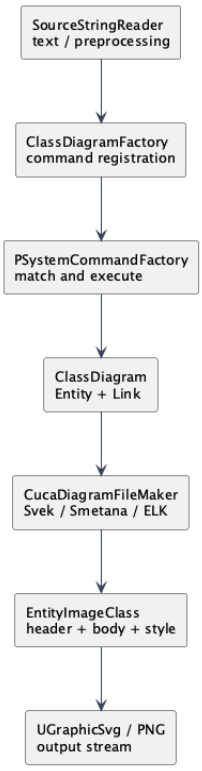


Figure 1: Class-diagram processing path. Each box identifies a responsibility, not a separate process.

```
@startuml
class Order
class Customer
Order --> Customer
@enduml
```

1. `SourceStringReader` preprocesses the text into diagram blocks. `ClassDiagramFactory` creates a `ClassDiagram` and registers commands.

2. `PSystemCommandFactory.getCandidate` tests registered commands against the current line. Its execution loop calls `AbstractDiagram.executeCommand`, which wraps execution in `ProtectedCommand` and delegates to the command.

3. `CommandCreateClass.executeArg` resolves the identifier in the diagram context, creates or updates an `Entity`, then attaches display and style data. `CommandLinkClass.executeArg` resolves the endpoints, constructs a `Link`, and adds it to the diagram. The checked model has two nodes and one link.

4. `CucaDiagram.getTextBlock` selects a layout backend. The normal desktop path can use Svek with Graphviz; the repeatable integration fixture adds `!pragma layout smetana` to select in-process Java layout.

5. `EntityImageClass` combines header and body blocks. Style signatures merge defaults and stereotype/style rules; explicit entity colors can override the relevant values. This is the point extended by the review palette.

6. `SourceStringReader.outputImage` calls the diagram's export method. `UgDiagram.exportDiagram` obtains a `TextBlockExporter`, which creates the format-specific drawing context and writes to the output stream. For SVG, group attributes reach `PortableSvgDocument`; PNG uses its raster drawing path.

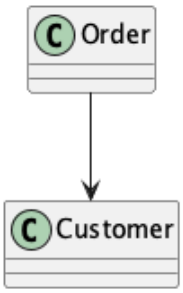


Figure 2: Baseline input rendered through the default desktop layout path with Graphviz installed.

Run results. Both complete source trees produced a JAR. Running `GPL_ONLY=1 bash gradlew jar test -Pfast -console=plain` on each tree reported 3,189 tests: 3,177 passed, 2 failed, 10 skipped. The JUnit XML and generated HTML report count 3,191 entries (3,177 passed, 2 failed, 12 skipped), including two additional skipped entries. Both trees show the same discrepancy; these totals are identified by evidence source rather than merged. Both failures are in `PicowebTest`: `get_png_bad_graphviz` and `get_png_svg_graphviz`, each expecting HTTP 200 but receiving 503. The same failures occur before and after the changes. Their cause is not resolved here; the complete suite is not described as passing. The focused baseline checks passed 3/3 and the final full-JAR integration checks passed 20/20. Commands and raw logs are linked from `README.md`.

2 Part 1: Pattern discovery

2.1 Method and counting rules

Candidate discovery began with source searches, but a class name was not accepted as proof. Each confirmed record requires an intent, a shared contract, identifiable participants and a call or ownership relationship that explains the behavior. The source mapping is commit-specific. Runtime tests establish the selected input path; source-only claims for optional backends are explicitly marked.

One record represents one collaboration. Multiple concrete commands do not become multiple Command instances. The baseline has five distinct pattern types and five instances. It maps 13 concrete classes in the varying GoF roles and 30 unique participant symbols including shared abstractions, clients and receiver objects. The concrete count includes concrete products for Factory Method and the selected composites/leaf for Composite; it excludes fixed contexts and assembly helpers. These are counts of mapped representatives, not of every implementation in PlantUML.

Pattern	Collaboration	Instances	Concrete
Command	Class diagram command dispatch	1	3
Factory Method	Class diagram product creation	1	2
Strategy	Class diagram layout backend selection	1	4
Composite	Class header text-block composition	1	3
Adapter	VizJs output-stream bridge	1	1

Two observations are rejected: Entity/Link membership alone does not prove Composite, and same-interface forwarding by UGraphicDelegator does not prove Adapter. One older candidate, text-block PlacementStrategy, remains uncertain within this input trace because its context was not fully mapped. It is excluded from confirmed counts. Template Method, Builder and graphics decorators are supporting idioms outside this five-type inventory; no absence claim is made.

Reading the mapping tables. All symbols below are actual source symbols. Except `net.atmp.CucaDiagram`, they use the prefix `net.sourceforge.plantuml.` The source column omits `Part1/code/src/main/java/net/sourceforge/plantuml/`; full paths and complete line ranges are in the CSV. A range spans the participant declaration and body; the named methods identify the relevant behavior.

2.2 Command: Class diagram command dispatch

Category and origin. GoF behavioral pattern; baseline application code.

Intent. Encapsulate recognized source instructions as interchangeable executable objects, separating command selection from model mutation.

Role	Actual symbol, methods and source
Command	`command.Command` isValid; execute `command/Command.java`:42–58
Shared command implementation	`command.SingleLineCommand2` execute; executeArg `command/SingleLineCommand2.java`:51–193
ConcreteCommand	`classdiagram.command.CommandCreateClass` getRegexConcat; executeArg `classdiagram/command/CommandCreateClass.java`:71–247
ConcreteCommand	`classdiagram.command.CommandLinkClass` executeArg; addLink `classdiagram/command/CommandLinkClass.java`:73–553
ConcreteCommand	`command.CommandPragma` executeArg `command/CommandPragma.java`:52–139
Invoker/selector	`command.PSystemCommandFactory` getCandidate; executeFewLines `command/PSystemCommandFactory.java`:61–309
Execution bridge	`core.AbstractDiagram` executeCommand `core/AbstractDiagram.java`:67–273
Client/registration	`classdiagram.ClassDiagramFactory` initCommandsList `classdiagram/ClassDiagramFactory.java`:88–173
Receiver	`classdiagram.ClassDiagram` inherited reallyCreateLeaf; addLink `classdiagram/ClassDiagram.java`:49–101
Receiver base	`objectdiagram.AbstractClassOrObjectDiagram` class diagram shared model `objectdiagram/AbstractClassOrObjectDiagram.java`:60–365
Model element	`abel.Entity` setColors; setStereotype `abel/Entity.java`:89–775
Model relationship	`abel.Link` constructor `abel/Link.java`:65–580

Evidence and interpretation. PSystemCommandFactory.getCandidate selects a Command using isValid; executeFewLines delegates through AbstractDiagram.executeCommand. CommandCreateClass.executeArg creates or updates an Entity; CommandLinkClass.executeArg creates a Link and adds it to the receiver. CommandPragma handles the layout directive in the traced input. No undo/history support is claimed.

net/sourceforge/plantuml/command/PSystemCommandFactory.java, lines 230–239

```
for (Command cmd : cmds) {
        final CommandControl result = safeIsValid(cmd, single);
        if (result == CommandControl.OK) {
                it.next();
                return new Step(cmd, single);
        }
        if (result == CommandControl.OK_PARTIAL) {
                final IteratorCounter2 cloned = it.cloneMe();
                final BlocLines lines = isMultilineCommandOk(cloned, cmd);
                if (lines == null)
```

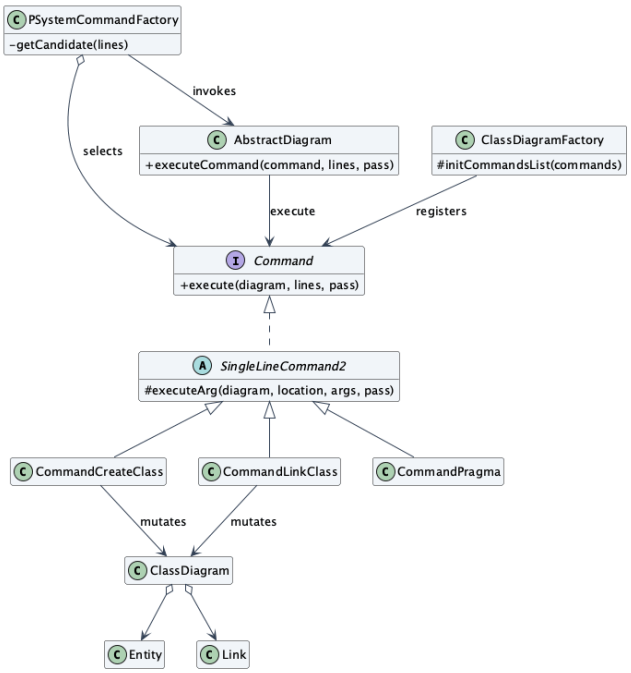


Figure 3: Class diagram command dispatch. Abbreviated names correspond to the mapping above.

Adding syntax changes the relevant command rather than the parser loop. The receiver still exposes a substantial mutable model API, so Command does not eliminate parser-to-model coupling. The chosen commands are stateless handlers; there is no claim that they support undo or replay history.

Verification. Confirmed by source inspection in this automated run. Human reviewer pending. Editable UML and PNG/SVG outputs: `Part1/diagrams/command.puml`. Detailed evidence: `Part1/evidence/command.md`.

2.3 Factory Method: Class diagram product creation

Category and origin. GoF creational pattern; baseline application code.

Intent. Let a specialized parser factory choose the diagram product while the superclass retains the parsing lifecycle.

Role	Actual symbol, methods and source
Creator	`command.PSystemCommandFactory` createSystem; createEmptyDiagram `command/PSystemCommandFactory.java`:61–309
ConcreteCreator	`classdiagram.ClassDiagramFactory` createEmptyDiagram `classdiagram/ClassDiagramFactory.java`:88–173
Product	`core.AbstractDiagram` executeCommand; exportDiagram contract `core/AbstractDiagram.java`:67–273
ConcreteProduct	`classdiagram.ClassDiagram` constructor `classdiagram/ClassDiagram.java`:49–101

Evidence and interpretation. PSystemCommandFactory.createSystem calls the abstract createEmptyDiagram hook. ClassDiagramFactory overrides it with a covariant ClassDiagram return and constructs that product. This is a virtual creation hook used by superclass code, not merely a helper named factory.

net/sourceforge/plantuml/classdiagram/ClassDiagramFactory.java, lines 94–99

```
    @Override
    public ClassDiagram createEmptyDiagram(PathSystem pathSystem, UmlSource source, Previous
previous,
                    PreprocessingArtifact preprocessing) {
            return new ClassDiagram(source, previous, preprocessing);
    }
```

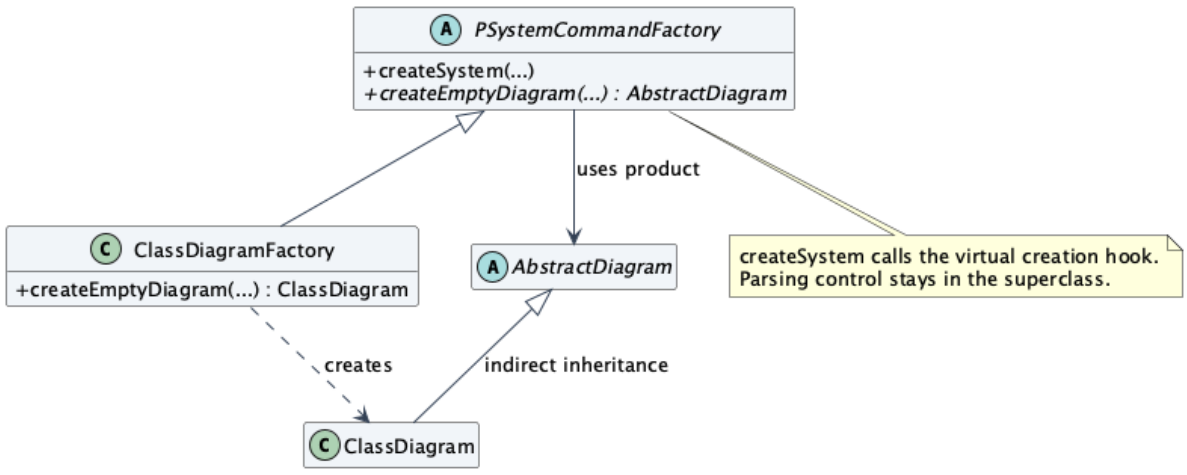


Figure 4: Class diagram product creation. Abbreviated names correspond to the mapping above.

The factory method preserves a common parse lifecycle across diagram families. It relies on inheritance at the creation boundary. A static helper returning a new ClassDiagram would not have the

same subclass-controlled creation intent.

Verification. Confirmed by source inspection in this automated run. Human reviewer pending. Editable UML and PNG/SVG outputs: `Part1/diagrams/factory.puml`. Detailed evidence: `Part1/evidence/factory.md`.

2.4 Strategy: Class diagram layout backend selection

Category and origin. GoF behavioral pattern; baseline application code.

Intent. Select a layout algorithm without forcing the caller to know each backend's graph processing steps.

Role	Actual symbol, methods and source
Context	`net.atmp.CucaDiagram` getTextBlock; dotIsAvailable `src/main/java/net/atmp/CucaDiagram.java`:109–959
Strategy	`svek.CucaDiagramFileMaker` getTextBlock `svek/CucaDiagramFileMaker.java`:46–71
ConcreteStrategy	`svek.CucaDiagramFileMakerSvek` getTextBlock `svek/CucaDiagramFileMakerSvek.java`:52–116
ConcreteStrategy	`sdot.CucaDiagramFileMakerSmetana` getTextBlock `sdot/CucaDiagramFileMakerSmetana.java`:116–925
ConcreteStrategy	`elk.CucaDiagramFileMakerElk` getTextBlock `elk/CucaDiagramFileMakerElk.java`:138–501
ConcreteStrategy	`svek.CucaDiagramFileMakerTeaVM` getTextBlock `svek/CucaDiagramFileMakerTeaVM.java`:56–148

Evidence and interpretation. CucaDiagram.getTextBlock selects a CucaDiagramFileMaker based on TeaVM, ELK, Smetana and Graphviz availability, then delegates to getTextBlock with the same arguments. Svek uses the external dot pipeline; Smetana performs layout in Java. ELK and TeaVM branches are source-verified but not exercised by the A17 tests.

net/atmp/CucaDiagram.java, lines 478–485

```
else if (this.isUseElk())
        maker = new CucaDiagramFileMakerElk(this);
else if (this.isUseSmetana() || this.dotIsAvailable() == false)
        maker = new CucaDiagramFileMakerSmetana(this);
else
        maker = new CucaDiagramFileMakerSvek(this);

return maker.getTextBlock(getDotStrings(), fileFormatOption);
```

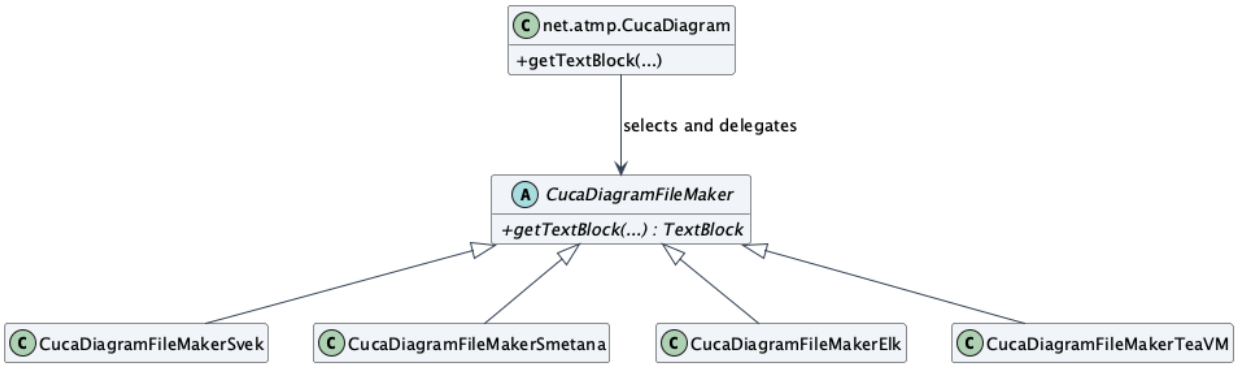


Figure 5: Class diagram layout backend selection. Abbreviated names correspond to the mapping above.

The selected object is local to getTextBlock; Strategy does not require a long-lived field. Selection remains a conditional in the context, so registering a completely new backend still requires editing that selection. Existing backends share the TextBlock result contract.

Verification. Confirmed by source inspection in this automated run. Human reviewer pending. Editable UML and PNG/SVG outputs: `Part1/diagrams/strategy.puml`. Detailed evidence: `Part1/evidence/strategy.md`.

2.5 Composite: Class header text-block composition

Category and origin. GoF structural pattern; baseline application code.

Intent. Treat a single drawable and an aggregate of drawables through the same measurement and drawing interface.

Role	Actual symbol, methods and source
Component	`klimt.shape.TextBlock` calculateDimension; inherited drawU `klimt/shape/TextBlock.java`:47–67
Composite base	`klimt.shape.TextBlockMemoized` calculateDimension; calculateDimensionSlow `klimt/shape/TextBlockMemoized.java`:50–72
Composite	`klimt.shape.TextBlockHorizontal` calculateDimensionSlow; drawU `klimt/shape/TextBlockHorizontal.java`:48–95
Composite	`klimt.shape.TextBlockVertical` calculateDimensionSlow; drawU `klimt/shape/TextBlockVertical.java`:51–134
Representative leaf	`klimt.shape.CircledCharacter` calculateDimension; drawU `klimt/shape/CircledCharacter.java`:46–88
Client	`svek.image.EntityImageClassHeader` constructor `svek/image/EntityImageClassHeader.java`:74–279
Assembly helper	`klimt.shape.TextBlockUtils` mergeLR; mergeTB `klimt/shape/TextBlockUtils.java`:55–183

Evidence and interpretation. TextBlockHorizontal and TextBlockVertical hold lists of TextBlock children, recursively measure them, and draw each through the component interface with translated coordinates. CircledCharacter is the inspected leaf. EntityImageClassHeader builds the header from smaller blocks through TextBlockUtils. The entity/link graph itself is not the evidence for Composite.

net/sourceforge/plantuml/klimt/shape/TextBlockHorizontal.java, lines 68–77

```
@Fast
public XDimension2D calculateDimensionSlow(StringBounder stringBounder) {
        XDimension2D dim = blocks.get(0).calculateDimension(stringBounder);
        for (int i = 1; i < blocks.size(); i++)
                dim = dim.mergeLR(blocks.get(i).calculateDimension(stringBounder));

        return dim;
}

@Override
```

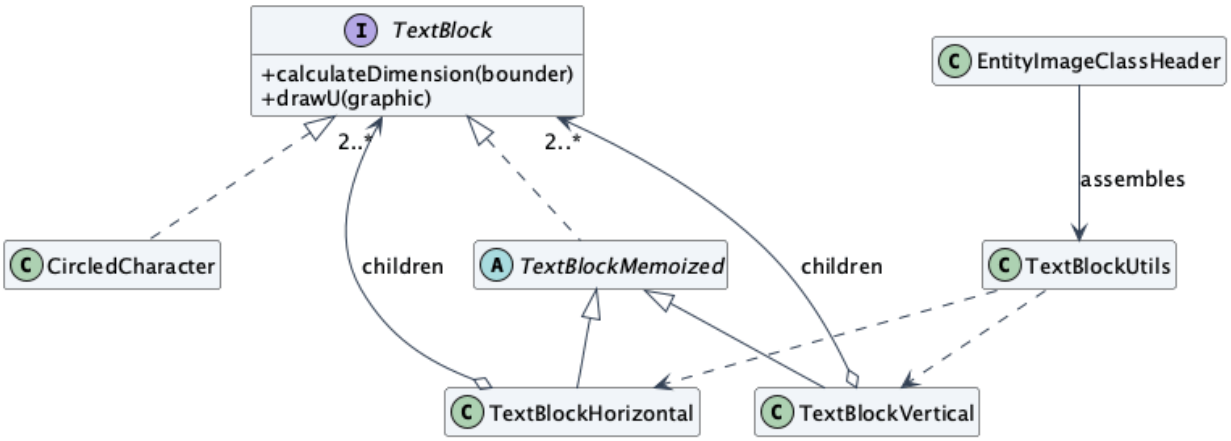


Figure 6: Class header text-block composition. Abbreviated names correspond to the mapping above.

A client can measure and draw an aggregate through TextBlock, and composites recursively invoke that same contract on their children. The selected leaf is CircledCharacter. Other rich-text leaves and margin wrappers exist; their complete implementation families are not included in the participant count.

Verification. Confirmed by source inspection in this automated run. Human reviewer pending. Editable UML and PNG/SVG outputs: `Part1/diagrams/composite.puml`. Detailed evidence: `Part1/evidence/composite.md`.

2.6 Adapter: VizJs output-stream bridge

Category and origin. GoF structural pattern; baseline application code.

Intent. Expose a string-returning JavaScript layout engine through PlantUML's existing Graphviz output-stream interface.

Role	Actual symbol, methods and source
Target	`dot.Graphviz` createFile3 `dot/Graphviz.java`:41–52
Adapter	`vizjs.GraphvizJs` createFile3; submitJob `vizjs/GraphvizJs.java`:52–153
Adaptee facade	`vizjs.VizJsEngine` execute `vizjs/VizJsEngine.java`:41–68
Client	`svek.DotStringFactory` getSvg; createFile3 call `svek/DotStringFactory.java`:70–654
Adapter selection	`dot.GraphvizRuntimeEnvironment` create `dot/GraphvizRuntimeEnvironment.java`:56–238

Evidence and interpretation. GraphvizJs implements Graphviz.createFile3(OutputStream). It submits a VizJsEngine.execute(dotString) operation, obtains its SVG String, writes bytes to the requested stream and returns ProcessState. VizJsEngine reflectively calls ch.braincell.viz.VizJS. This incompatible-interface conversion establishes Adapter intent. Optional VizJs execution is not claimed.

net/sourceforge/plantuml/vizjs/GraphvizJs.java, lines 88–100

```
	public GraphvizJs(String dotString) {
		this.dotString = dotString;
	}

	public ProcessState createFile3(OutputStream os) {
		try {
			final String svg = submitJob().get();
			os.write(svg.getBytes());
			return ProcessState.TERMINATED_OK();
		} catch (Exception e) {
			Logme.error(e);
			throw new GraphvizJsRuntimeException(e);
		}
```

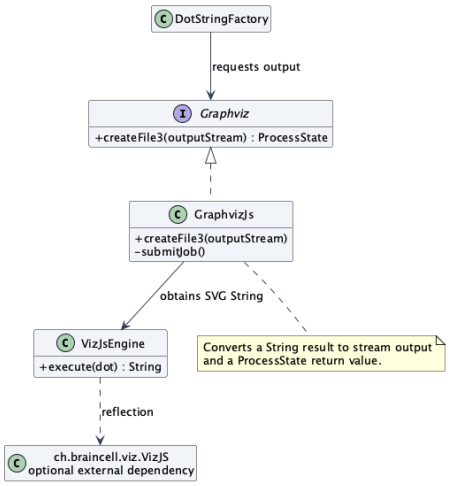


Figure 7: VizJs output-stream bridge. Abbreviated names correspond to the mapping above.

The mismatch is concrete: an engine returns a String, while the caller requires stream output and ProcessState. The adapter also manages asynchronous execution. This source evidence confirms the collaboration, but the optional ch.braincell VizJS dependency was not installed or executed.

Verification. Confirmed by source inspection in this automated run. Human reviewer pending. Editable UML and PNG/SVG outputs: `Part1/diagrams/adapter.puml`. Detailed evidence: `Part1/evidence/adapter.md`.

3 Part 2: Evolution and refactoring

The three changes form a small review workflow for class diagrams. Status is stored once in the model, rendered as a visible label, optionally mapped to a fill policy, and optionally exposed to SVG consumers. Existing tags and stereotypes can label classes, but do not provide this constrained property and its dedicated output contract. The implementation therefore extends existing mechanisms instead of claiming that annotations, colors or metadata were absent.

3.1 CH01: Explicit review status on a class declaration

Requirement. A single-line declaration accepts a final `[review=pending]`, `[review=approved]` or `[review=rejected]` option. Matching is case-insensitive. Omitted status preserves the current value on redeclaration; an explicit status replaces it. The label appears in the class header. Existing class/link topology is unchanged. Unknown values must produce a parse error.

```
@startuml
!pragma layout smetana
class Order [review=pending]
class Customer [review=approved]
Order --> Customer
@enduml
```

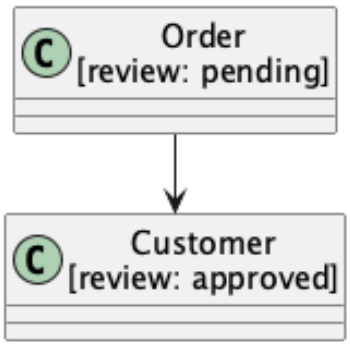


Figure 8: CH01: explicit review labels on the two class headers.

Impact. `CommandCreateClass.getRegexConcat` recognizes the suffix and `executeArg` stores it. `Entity` gains the property and accessors. `EntityImageClassHeader` adds a display line before building the name TextBlock. No dispatcher replacement or separate diagram subtype is needed.

Acceptance evidence. 11/11 focused checks passed, covering valid states, unknown-state rejection, model counts, redeclaration, explicit color coexistence and PNG/SVG output. The production API's setter is intentionally small; syntax validation occurs in the command. Multiline declaration syntax and text/XMI exports are not extended. The option can be declared on a separate line before adding members. This boundary is part of the requirement, not an untested claim that every serializer supports the new property.

Pattern effect. Command is modified in one concrete handler. Factory Method, layout Strategy, text-block Composite and VizJs Adapter remain preserved. The Composite tree receives different text; that does not create a second Composite collaboration. No unrelated refactoring is claimed for CH01.

3.2 CH02: Configurable review styling policy

Requirement. `!pragma classReviewPalette` accepts `default`, `pastel` or `mono`. Default retains normal style resolution. Pastel maps pending to `#FFF3CD`, approved to `#D4EDDA` and rejected to `#F8D7DA`; mono uses `#EEEEEE` for all reviewed nodes. Explicit class fill wins over the policy. Unreviewed nodes retain existing styles. The policy changes fill only; it is not a guarantee of accessibility for arbitrary custom fonts and themes.

```
@startuml
!pragma layout smetana
!pragma classReviewPalette pastel
class Order [review=pending]
class Customer [review=approved]
Order --> Customer
@enduml
```

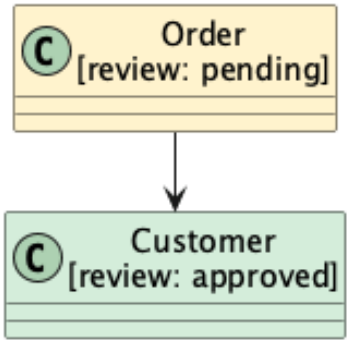


Figure 9: CH02: state-dependent pastel fill. Text labels remain useful when colors are unavailable.

Impact. `CommandPragma` validates the option and `PragmaKey` records it. `EntityImageClass.drawInternal` reads the existing review property and asks a `ReviewPalette` for its fill. The nested `Pastel` and `Mono` classes implement the same operation. The client tests whether an explicit color already exists before applying the policy.

Acceptance evidence. 15/15 cumulative checks passed. Both policy choices, invalid-value rejection and explicit-color precedence are tested. A failed early assertion was corrected after observing that the SVG writer abbreviates `#EEEEEE` to `#EEE`; the color itself was correct.

Pattern effect. A new Strategy instance, *Review-state fill policy*, is introduced. The original layout Strategy keeps its name and count. Command is modified to validate the pragma. Other mapped collaborations are preserved. The small interface makes policy behavior independently replaceable, but the selector still enumerates policy names. Two classes are a deliberate trade-off for the course experiment; this is not evidence that every two-color rule needs a Strategy hierarchy.

3.3 CH03: Optional SVG metadata and a local refactoring

Requirement. `!pragma svgReviewMetadata true` adds `data-review-status` to reviewed entity groups. The default and explicit `false` omit it. Unreviewed entities do not receive an empty attribute. Only true/false are accepted. The diagram's visible output is unchanged.

```
@startuml
!pragma layout smetana
!pragma svgReviewMetadata true
class Order [review=pending]
class Customer [review=approved]
```

```
Order --> Customer
@enduml
```

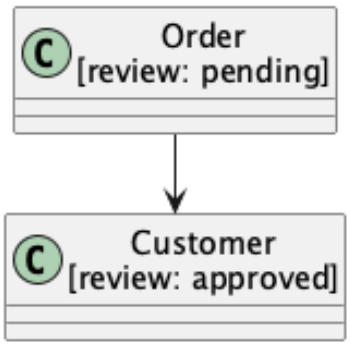


Figure 10: CH03: the visible diagram is unchanged; additional information is in SVG group attributes.

```
<g class="entity" ... data-review-status="pending" ...>
  <!-- Order shapes and label -->
</g>
```

The fragment above abbreviates unrelated SVG attributes. The complete output is `Part2/evidence/CH03/metadata.svg`.

Impact. The validated pragma is read by `EntityImageClass`, which adds a `UGroupType.DATA_REVIEW_STATUS` entry. The portable SVG serializer must explicitly allow that key. The first implementation reached the group but failed the integration test because the serializer ignored the unfamiliar key. Extending the allow-list fixed the defect. This shows why tracing only parser and model code would have missed a real output integration requirement.

Refactoring. `drawInternal` previously requested the same merged style separately for shadow, border, background and stroke. A local `resolvedStyle` now supplies those values. This is removal of redundant style lookups, not a pattern replacement. The recorded CH02 and CH03 `review.svg` outputs are byte-identical with metadata disabled.

Acceptance evidence. 20/20 cumulative checks passed, including enabled, default-off, explicit-off, missing-status and invalid-value cases. The same 20 checks passed against the JAR produced by the complete modified source build. All format-specific claims are limited to SVG metadata and the PNG/SVG visible outputs exercised here. No new file format or sidecar is claimed.

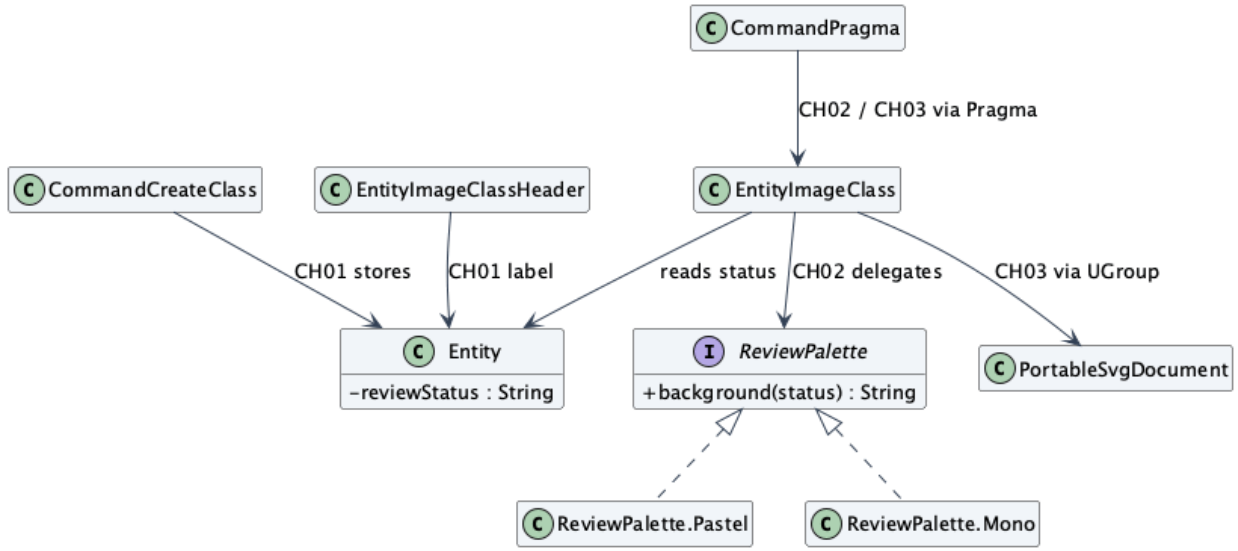


Figure 11: Changed responsibilities across parser, model, styling and SVG output.

3.4 Snapshots, metrics and pattern effects

Each change has a separate source commit, cumulative source snapshot, patch and test log in `Part2/changes/Change1`, `Change2` and `Change3`. The patches apply consecutively to the recorded baseline. Commit IDs identify the local experiment; the patch and saved source allow reproduction without requiring those student commits to be published upstream.

Snapshot	Exact commit
Baseline	`149874a1bb2b64c42889d7178cb89f6647ac14bc`
CH01	`8e9f313b8f589012115d631df4aad2a369a22f53`
CH02	`0d170a89f61fc811f729f236a28634c697f985a6`
CH03	`dd5c91b457c13fb1a20ae51a62fa73c3b8485847`

Snapshot	LOC	Max CC	Types	Instances	Focused tests
Baseline	1314	20	5	5	3/3
CH01	1329	20	5	5	11/11
CH02	1366	20	5	6	15/15
CH03	1376	20	5	6	20/20

LOC and cyclomatic complexity use Lizard 1.24.1 on the same union of nine production Java paths at every snapshot. The not-yet-existing policy file contributes zero at baseline/CH01. LOC excludes blank/comment lines and tests. Maximum function CC follows Lizard's language parser and decision-token rules; per-function evidence and exact measurement scope are saved. Maximum CC stays 20 because an existing function dominates the scope. This does not mean every changed function has unchanged complexity.

The 62-line net increase buys a property, validation, visible annotation, two policies and an output contract. It does not by itself demonstrate better design. The stronger evidence is that parser dispatch, product creation and layout contracts did not have to change. Conversely, the SVG allow-list remains a central extension point, and palette selection still requires an explicit choice.

Collaboration	CH01	CH02	CH03
Command dispatch	Modified	Modified	Modified
Product creation	Preserved	Preserved	Preserved
Layout backend selection	Preserved	Preserved	Preserved
Text-block composition	Preserved	Preserved	Preserved
VizJs output bridge	Preserved	Preserved	Preserved
Review-state fill policy	Absent	New	Preserved

There are no replaced or removed collaborations in these changes. Concrete mapped variable participants rise from 13 to 15. Unique participant symbols rise from 30 to 34: the palette interface, two nested implementations and the newly mapped EntityImageClass context. Rejected/uncertain observations remain 2/1. The CSV retains continuing instance names and counts each source version separately.

4 Part 3: Cross-language comparison

4.1 Selected problems and equivalent behavior

Command-driven model construction and selectable rendering are the two primary problems. The instruction sheet additionally asks for one representative of every confirmed type, so the same small program also includes the creation hook, nested block composition and a legacy-renderer adapter. This produces five pattern rows per language in the dataset without maintaining twenty duplicate programs.

The miniature DSL accepts `class NAME`, `link LEFT RIGHT`, `group` and `end`. Names are nonempty printable ASCII tokens without whitespace. Blank lines are ignored. Names must be globally unique and link endpoints must already exist. Self-links are allowed. Groups may nest and flatten in insertion order. Wrong arity, unknown commands, duplicate names, missing endpoints and unmatched/unclosed groups return specified error messages. This is a teaching model, not a second PlantUML parser.

```
class Order
class Customer
link Order Customer

compact   -> Order,Customer|Order->Customer
bracketed -> [Order],[Customer]|Order->Customer
json      -> ["Order","Customer"]
```

The JSON names backend intentionally emits names only; its contract is a legacy list serializer, not a lossless graph export. The adapter extracts that list from the diagram and converts the interface expected by the exporter.

Pattern	Shared miniature participants
Command	Command; Create, Link, OpenGroup, CloseGroup; Parser invoker; Diagram receiver
Factory Method	Parser creation hook; DiagramParser creator; Model/Diagram product hierarchy
Strategy	Exporter context; Renderer contract; Compact and Bracketed renderers
Composite	Block; Node leaf; Group containing Block children recursively
Adapter	Renderer target; NamesAdapter; LegacyNames.encode(names) adaptee

Behavioral evidence. Sixteen shared fixtures passed in each language: 64 successful checks in total. Fixtures include empty input, simple/link models, both renderers, nested/empty groups, duplicate and missing names, invalid syntax, group errors, whitespace, self-links and quote/backslash JSON escaping. Production code and command-line runners are separate. Tests and expected results are in `Part3/test-cases/fixtures.json`; `Part3/run_tests.py` compiles Java/C++ and runs all four implementations against the same file.

4.2 Static and process measurements

Language	LOC	Max CC	Cases	Median ms	Toolchain
Java	106	6	16/16	27.206	OpenJDK 25.0.2
Python	101	6	16/16	16.827	CPython 3.14.2
JavaScript	97	6	16/16	21.728	Node.js 22.19.0
C++	156	7	16/16	2.279	Apple clang 21; C++17

Production LOC excludes runner files, tests, blank lines and comments. Lizard 1.24.1 measures

source LOC and maximum function CC. Its per-language parsing means the C++ input-tokenization loop contributes an additional branch in this implementation; this is not a universal property of C++.

Java and C++ were compiled before timing. Each language ran one unmeasured warm-up process and five fresh measured processes on the same three-line link fixture. The median includes process startup, imports/class loading, standard input, parsing and output capture; it excludes compilation and the sixteen-case test loop. Raw samples are in `Part3/evidence/results.json`. These tiny inputs are dominated by startup. They do not support conclusions about steady-state renderer speed. Measurements are repeated on the five instance rows in `data/cross_language.csv`; they must not be added together.

4.3 Structure, coupling and language features

All four implementations retain the same 18 domain types and 11 domain inheritance/implementation relationships (excluding Python's ABC library base). Five roles are abstract contracts: Block, Model, Command, Parser and Renderer. Java expresses four as interfaces and one as an abstract class. Python uses ABC and abstractmethod. JavaScript uses ordinary base classes whose default methods throw, so abstractness is a convention checked at runtime. C++ uses pure virtual operations and virtual destructors. The classes are explicit to make comparison possible; shorter function-based Python or JavaScript designs would be reasonable but would change the structural comparison.

Coupling. Concrete commands depend on Diagram operations, while the parser owns the command registry. Exporter depends on Renderer and does not name the concrete renderers. NamesAdapter knows both the diagram representation and the legacy list serializer; that knowledge is localized. Group depends on Block, so nesting does not add a new rendering branch. These boundaries are equivalent across languages, even where the language does not statically enforce them.

Inheritance and composition. Inheritance supplies replaceable operations and the factory hook. Composition supplies the command registry, child blocks, selected renderer and wrapped legacy service. C++ makes ownership explicit with unique_ptr; the group stack contains non-owning pointers to heap-owned nodes. Java, Python and JavaScript leave memory reclamation to their runtimes. The C++ Exporter holds a non-owning renderer reference whose lifetime is controlled by the runner. The other versions hold managed object references.

Technology discussion. Java's interfaces and abstract classes provide compile-time contracts and shared implementation respectively [3]. Python's ABC module permits similar explicit contracts while normal dispatch remains dynamic [4]. JavaScript class syntax operates on its prototype-based object system [5]. The C++ design follows explicit ownership and RAII guidance [6]. These are language mechanisms supporting pattern intent; none proves a pattern solely because inheritance is present. Command and Adapter are covered directly in Unit 2; the cohesion/coupling discussion connects to GRASP in Unit 3 without claiming an additional measured GRASP inventory.

5 LLM records and human verification

This report and code were prepared with AI assistance in the actual interaction recorded on 2026-10-11. The session metadata identifies `gpt-6-astra`. The displayed UI label and access tier were not independently verified; the report does not assert that the LLM run satisfies the course's free/public-tool condition. PlantUML and the compilation/testing tools used here are publicly available.

`prompts/exact_user_prompts.txt` contains the supplied prompt and clarification. `prompts/raw_public_outputs.jsonl` captures actual assistant messages and tool-call text, including code-generation operations. It is an interaction prefix, not a fabricated set of separate trials. The input manifest identifies the PDFs, image, baseline source and earlier draft. System instructions and private reasoning are not included. Each part is mapped to this real interaction in `data/llm.csv`.

Claim or output	Evaluation	Correction/evidence
First source hash described as commit	Partially correct	It was a tag object; peeled commit recorded after clone
Old placement Strategy inventory	Cannot confirm within trace	Excluded; concrete layout context mapped instead
Initial SVG metadata patch	Incorrect/incomplete	Allow-list omission detected and repaired
Mono color test assertion	Incorrect assertion	Valid #EEE output inspected; test corrected
Final evolution behavior	Automatically verified for scope	20/20 focused checks on full built JAR
Four-language equivalence	Automatically verified for fixtures	16/16 per language; raw outputs saved

Human verification is pending. No reviewer name or student contribution has been invented. `data/human_verification.csv` provides a place to record the actual reviewer, date, evidence and correction. A human review should verify the source-role mappings, rerun the supplied tests, inspect the three rendered examples and explain why adding the SVG group key alone was insufficient.

6 Findings, contributions and references

6.1 Findings and limitations

The strongest evidence for the patterns lies in calls and delegation. Commands encapsulate mutations; the factory method controls product creation; layout strategies expose a common result; text-block composites recurse through one contract; the VizJs adapter converts a real interface mismatch. The changes preserve those existing boundaries. A new policy Strategy adds one collaboration without adding a new pattern type.

The useful extension points are not equally open. A new command option fits the existing handler, but a new SVG attribute must also pass a serializer allow-list. The review policy isolates variable fills, yet its name selector remains central. The refactoring removes repeated style lookups for one draw call; the experiment does not establish a measurable performance improvement.

Limitations include the representative rather than exhaustive scope, source-only verification of VizJs/ELK/TeaVM branches, the two unchanged upstream test failures, single-line review syntax, and metadata limited to SVG. Human verification, faculty approval, completion of the assigned student contributions and LLM-tier confirmation remain outside the automated evidence. These limits are explicit so the submission does not present unperformed work as an experiment.

6.2 Contributions

The following division assigns technical work that each student can personally verify and correct. It is a responsibility record, not a claim that the checks have already been performed. The exact tasks and sign-off fields are in `HUMAN_VERIFICATION.md` and `data/human_verification.csv`.

Sathvik Pasuvula (AM.SC.U4CSE23064, CSE A). Own the baseline class-diagram trace and inspect the Command, Factory Method and Composite participant mappings against the pinned Java source. Reproduce CH01 from its patch, check the parser-to-`Entity`-to-header path and the 11 focused checks, then run and review the Java and Python fixture implementations.

P Sai Satvik Chowdary (AM.SC.U4CSE23142, CSE A). Own the layout Strategy and VizJs Adapter source review. Reproduce CH02 and CH03, check palette precedence and the SVG serializer allow-list, inspect the generated diagrams and 15/20 focused checks, then run and review the JavaScript and C++ implementations and reconcile the CSV metrics with the workbook.

Each student should record the date, source lines or commands checked, result and corrections before this section is presented as completed contribution. The generated source, tests and report remain disclosed as AI-assisted work in the LLM record; providing the assignment brief and performing an actual source review are different contributions.

6.3 References

1. PlantUML, v1.2026.8 source and class-diagram documentation. https://github.com/plantuml/plantuml/tree/v1.2026.8; https://plantuml.com/class-diagram. Source citations throughout use the pinned commit.

2. E. Gamma, R. Helm, R. Johnson and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1994, ISBN 9780201633610. Publisher record: https://www.informit.com/store/design-patterns-elements-of-reusable-object-oriented-software-9780201633610.

3. Oracle, Abstract Methods and Classes. https://docs.oracle.com/javase/tutorial/java/IandI/abstract.html.

4. Python documentation, abc: Abstract Base Classes. https://docs.python.org/3/library/abc.html.

5. MDN, JavaScript Classes. MDN JavaScript classes reference.

6. C++ Core Guidelines, resource management and ownership. C++ Core Guidelines (online).

7. Lizard 1.24.1, source-code metrics tool. https://github.com/terryyin/lizard. The supplied metric scripts and raw function records define the actual measurement used here.

Online references accessed 2026-10-11. Language comparisons are supported by the local implementations and tests, not by benchmark claims from these documents.

7 Team folder and datasets

One report and one authoritative dataset folder are supplied. Part folders hold code and evidence, not separate report submissions. The instruction sheet's dataset names take precedence over inconsistent example capitalization and typos in its folder tree. The Excel workbook presents the same observations as the CSVs.

```
A17/
  README.md, HUMAN_VERIFICATION.md
  A17_Report.tex, A17_Report.pdf
  data/  CSVs, A17_Dataset.xlsx, test_result/
  Part1/  code/ (baseline subset), diagrams/, evidence/
  Part2/  code/ (evolved subset), changes/, diagrams/,
          tests/, evidence/, reconstruct.sh, run_evolution.py
  Part3/  diagram/{java,python,javascript,cpp}/,
          test-cases/, evidence/, A17_slide.pdf, run_tests.py
  prompts/  exact prompts, public outputs, verification
  tools/  validation script
```